CORE103 Computational Thinking
PW6 - Strings



Department of Computer Science

Dhanani School of Science and Engineering

Habib University

Fall 2026

Copyright @ 2026 Habib University

Contents

1 Introduction **2**
1.1 Instructions 2
1.1.1 Lab Objectives 2

2 Prerequisites **3**
2.1 Assert Statements 3
2.2 Pytest 3

3 Exercises **5**
3.1 Campus Code Mirror 5
3.2 Scavenger Hunt Decoder 6
3.3 Game of Letters 8
3.4 Binary Tile Challenge 9
3.5 Encryption Check 10
3.6 Potato Song Generator 12
3.7 Not Bad 13
3.8 Student Email Generator 14

Introduction

1.1 Instructions

- This PW must be submitted online via CANVAS. You are required to submit a zip file that contains all the `.py` files.
- The zip file should be named as `PW6_aa123456.zip` where `aa123456` will be replaced with your student id.

1.1.1 Lab Objectives

The following are the lab objectives of this lab:

- Iterate over and manipulate strings in Python.
- Use string indexing and slicing where appropriate.
- Build and transform strings based on specified rules.
- Choose suitable approaches for solving string-processing problems.
- Parse structured information contained within strings.
- Debug programs involving string processing and iteration.

Prerequisites

2.1 Assert Statements

Assert statements are crucial tools in programming used to verify assumptions and test conditions within code. For instance, in Python, an assert statement is typically as follows:

```
assert condition, message
```

For example, in a function calculating the square root, in order to ensure that the input value is non-negative, one might add the following assert statement:

```
assert x >= 0, 'x should be a non-negative number'
```

If the condition evaluates to False, an AssertionError is raised, highlighting the issue and its corresponding message.

2.2 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
pip install pytest
```

After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
pytest
```

If you want to test any particular file, then you will have to specify the filename as follows:

```
pytest <filename>.py
```

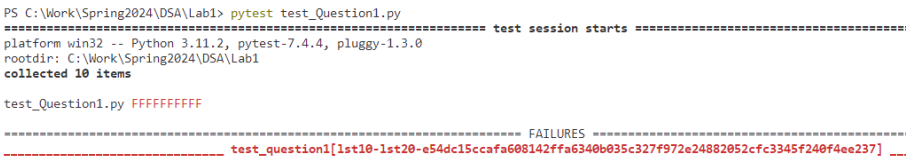
If all the tests pass successfully, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
============================= test session starts ========
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py ..........

============================== 10 passed in 0.06s =========
```

However, if one or more tests fail, you should receive a similar output.



Exercises

3.1 Campus Code Mirror

This question should be implemented in the file ***q1.py****.*

During a campus treasure hunt, each team receives a secret message. To hide the message, every English letter is replaced with its **mirror letter** in the alphabet. The first letter is paired with the last, the second with the second-last, and so on.

For example:

```
A -> Z     B -> Y     C -> X
a -> z     b -> y     c -> x
```

Uppercase letters remain uppercase and lowercase letters remain lowercase. Digits, spaces, and special characters remain unchanged.

Given a message, generate its mirrored version.

Using ord() and chr()

Python provides the functions **ord()** and **chr()** for working with character codes. The function **ord()** converts a character to its numeric code, while **chr()** converts a numeric code back to a character.

```
>>> ord('A')
65

>>> ord('Z')
90

>>> chr(65)
'A'

>>> chr(90)
'Z'
```

Constraints

- `isinstance(message, str)` is `True`.
- $1 \leq$ `len(message)` ≤ 1000
- Use **iteration**.

Function Specification

Write an **iterative** function named **mirror_message** that takes one parameter, **message**, and returns the mirrored message.

Sample Interaction

```
>>> mirror_message('ABC')
'ZYX'

>>> mirror_message('abc')
'zyx'

>>> mirror_message('Habib')
'Szyry'

>>> mirror_message('CS101!')
'XH101!'
```

Testing

To test your function, type:

```
pytest tests/test_q1.py
```

3.2 Scavenger Hunt Decoder

This question should be implemented in the file ***q2.py****.*

A student society is running an orientation-week scavenger hunt. Each clue is a scrambled string of characters, such as `'2qc2a3zt'`. The digits in the string are **jump sizes** that reveal the hidden message.

Start at index `0` and scan encoded_text from left to right. Whenever you encounter a digit, convert it to a jump size and move forward by that many positions from your current position. The character you land on becomes the next character of the hidden message.

Every character in the string, including digits, counts as a position when jumping. A jump of `0` selects the character at the current position.

For example, if `encoded_text` is `'2qc2a3zt'`:

```
start at index 0
jump 2 -> 'c'
jump 2 -> 'a'
jump 3 -> 't'
hidden message -> 'cat'
```

If the string contains no digits, the hidden message is the empty string `''`.

Constraints

- `isinstance(encoded_text, str)` is `True`.
- $1 \leq$ `len(encoded_text)` ≤ 1000
- Each digit represents a jump size from `0` to `9`.
- Every jump remains within the bounds of encoded_text.
- Use **iteration**.

Function Specification

Write an **iterative** function named **decode_message** that takes one parameter, **encoded_text**, and returns the hidden message.

Sample Interaction

```
>>> decode_message('2qc2a3zt')
'cat'

>>> decode_message('xgo11')
'go'

>>> decode_message('H0zi3')
'Hi'

>>> decode_message('1a2 2b')
'a b'

>>> decode_message('habib')
''
```

In the first case, the jump sizes are `2`, `2`, and `3`. Starting from index `0`, they land on `'c'`, `'a'`, and `'t'`, giving `'cat'`.

In `'1a2 2b'`, the jumps land on `'a'`, a space, and `'b'`, so the hidden message is `'a b'`.

In `'habib'`, there are no digits, so the function returns `''`.

Testing

To test your function, type:

```
pytest tests/test_q2.py
```

3.3 Game of Letters

This question should be implemented in the file ***`q3.py`****.*

Ahmed and Bareeha are playing a game using a collection of letter tiles. The tiles are represented by a string letters. Each player starts with an empty string: ahmed_string and bareeha_string.

Ahmed plays first, and the players then take alternate turns. On each turn, a player may choose **any one** of the remaining letters, add it to the end of their own string, and remove that occurrence from letters.

The game ends when no letters remain. The players win if ahmed_string and bareeha_string are identical.

Given letters, determine whether there exists a sequence of moves that allows both players to win.

Constraints

- `isinstance(letters, str)` is `True`.
- $1 \leq$ `len(letters)` ≤ 1000
- letters contains lowercase English letters only.
- Use **iteration**.

Function Specification

Write an **iterative** function named **can_form_same_word** that takes one parameter, **letters**. Return `True` if there exists a sequence of moves such that Ahmed and Bareeha finish with identical strings. Otherwise, return `False`.

Sample Interaction

```
>>> can_form_same_word('baba')
True

>>> can_form_same_word('cbcba')
False

>>> can_form_same_word('abca')
False
```

In the first case, Ahmed and Bareeha can both finish with the string `'ba'`, so the function returns `True`.

In the second and third cases, there is no sequence of moves that allows their final strings to be identical.

Testing

To test your function, type:

```
pytest tests/test_q3.py
```

3.4 Binary Tile Challenge

This question should be implemented in the file ***`q4.py`***.

At a campus coding challenge, Fatima is given a row of binary tiles represented by a string binary_string. Each tile contains either `'0'` or `'1'`. Her goal is to arrange all the `'0'`s before all the `'1'`s, giving the smallest possible binary value.

Fatima may only change the arrangement by reversing a consecutive section of the string. For example:

```
01001 -> 00011
```

This can be done by reversing the characters from index `1` up to, but not including, index `4`.

Given binary_string, determine the minimum number of substring reversals required so that all `'0'`s appear before all `'1'`s.

Constraints

- `isinstance(binary_string, str)` is `True`.
- $1 \leq$ `len(binary_string)` ≤ 1000
- binary_string contains only `'0'` and `'1'`.
- Use **iteration**.

Function Specification

Write an **iterative** function named **minimum_reversals** that takes one parameter, **binary_string**, and returns the minimum number of substring reversals required to place all `'0'`s before all `'1'`s.

Sample Interaction

```
>>> minimum_reversals('00111')
0

>>> minimum_reversals('111000')
1

>>> minimum_reversals('010101')
2
```

In the first case, all `'0'`s already appear before all `'1'`s, so no reversal is required.

In the second case, reversing the entire string produces `'000111'`, so only one reversal is required.

For `'010101'`, one possible minimum sequence is:

```
010101
001011
000111
```

Therefore, the minimum number of reversals is `2`.

Testing

To test your function, type:

```
pytest tests/test_q4.py
```

3.5 Encryption Check

This question should be implemented in the file ***`q5.py`***.

Sania is checking whether two messages follow the same character-replacement pattern. She receives an original string original_message and an encoded string encoded_message.

The two strings follow the same pattern if:

- every occurrence of the same character in original_message maps to the same character in encoded_message; and
- two different characters in original_message do not map to the same character in encoded_message.

A character is allowed to map to itself.

For this question, if a character appears more than once in either string, its repeated occurrences will appear next to each other.

Given the two strings, determine whether they follow the same pattern.

Constraints

- `isinstance(original_message, str)` and `isinstance(encoded_message, str)` are `True`.
- `len(original_message) == len(encoded_message)`.
- $1 \leq$ `len(original_message)` ≤ 1000
- If a character repeats in either string, its repeated occurrences are consecutive.
- Use **iteration**.

Function Specification

Write an **iterative** function named **same_pattern** that takes two parameters, **original_message** and **encoded_message**. Return `True` if the two strings follow the same pattern. Otherwise, return `False`.

Sample Interaction

```
>>> same_pattern('foo', 'bar')
False

>>> same_pattern('egg', 'add')
True

>>> same_pattern('ab', 'cc')
False

>>> same_pattern('aabb', 'ccdd')
True
```

In the first case, the two occurrences of `'o'` would need to map to different characters, so the strings do not follow the same pattern.

In the second case, `'e'` maps to `'a'` and both occurrences of `'g'` map to `'d'`, so the pattern is consistent.

In the third case, two different characters would both map to `'c'`, so the strings do not follow the same pattern.

Testing

To test your function, type:

```
pytest tests/test_q5.py
```

3.6 Potato Song Generator

This question should be implemented in the file ***q6.py****.*

The nursery rhyme "1 potato, 2 potato" begins with:

```
1 potato, 2 potato, 3 potato, 4!
5 potato, 6 potato, 7 potato, MORE!
```

Each verse contains two lines and uses seven consecutive numbers. The first line contains the first four numbers, while the second line contains the next three numbers followed by `MORE!`. The counting continues across verses.

Given the number of verses, generate the complete song.

Constraints

- `isinstance(verses, int)` is `True`.
- $1 \leq \textit{verses} \leq 100$
- Use **iteration**.

Function Specification

Write an **iterative** function named **potato_song** that takes one parameter, **verses**, and returns the complete song as a single string. Each line must be separated by a newline character, with no blank lines between verses.

Sample Interaction

```
>>> print(potato_song(1))
1 potato, 2 potato, 3 potato, 4!
5 potato, 6 potato, 7 potato, MORE!

>>> print(potato_song(3))
1 potato, 2 potato, 3 potato, 4!
5 potato, 6 potato, 7 potato, MORE!
8 potato, 9 potato, 10 potato, 11!
12 potato, 13 potato, 14 potato, MORE!
15 potato, 16 potato, 17 potato, 18!
19 potato, 20 potato, 21 potato, MORE!
```

For three verses, the song contains six lines. Each verse uses seven numbers, so the counting continues from 1 through 21.

Testing

To test your function, type:

```
pytest tests/test_q6.py
```

3.7 Not Bad

This question should be implemented in the file ***q7.py****.*

A student club is testing a simple positivity filter for comments. If some text contains `'not'` followed later by `'bad'`, the filter replaces that entire portion with `'good'`.

Find the first occurrence of `'not'` and then the first occurrence of `'bad'` that appears after it. The search ignores capitalization and works on substrings, so `'bad'` may also occur inside a longer word such as `'badminton'`.

Replace everything from the beginning of `'not'` up to and including `'bad'`. If the first character of the matched `'not'` is uppercase, use `'Good'` as the replacement. Otherwise, use `'good'`. If no suitable pair exists, return the original text unchanged.

Constraints

- `isinstance(text, str)` is `True`.
- $1 \leq$ `len(text)` ≤ 1000

Function Specification

Write a function named **replace_not_bad** that takes one parameter, **text**, and returns the modified text.

Sample Interaction

```
>>> replace_not_bad('This dinner is not that bad!')
'This dinner is good!'

>>> replace_not_bad('Maula Jatt is awesome.')
'Maula Jatt is awesome.'

>>> replace_not_bad('Cheating is not my cup of tea.')
'Cheating is not my cup of tea.'

>>> replace_not_bad('Babar was not a badminton player growing up
   in Islamabad.')
'Babar was goodminton player growing up in Islamabad.'

>>> replace_not_bad('Not every Badshah is a Maharaja.')
'Goodshah is a Maharaja.'
```

In the first case, `'not that bad'` is replaced by `'good'`.

In the fourth case, `'bad'` occurs at the beginning of `'badminton'`, so the matching portion is replaced.

In the final case, the search ignores capitalization. Since `'Not'` begins with an uppercase letter, the replacement is `'Good'`.

Testing

To test your function, type:

```
pytest tests/test_q7.py
```

3.8 Student Email Generator

This question should be implemented in the file ***`q8.py`****.*

You are helping the university IT department create email IDs for a new batch of students. Each student's ID is generated from their name and a running serial number.

An email ID is generated using the following rules:

- Use the first letter of the student's first name and the first letter of their last name.
- Convert both initials to lowercase.
- Ignore any middle names.
- Add a five-digit serial number starting from `00000`.
- End the ID with `@st.habib.edu.pk`.

For example, `'Tom Marvolo Riddle'` with serial number `2` produces:

```
tr00002@st.habib.edu.pk
```

The serial number increases by `1` after each ID is generated and starts again from `0` each time the function is called.

Constraints

- `isinstance(num_students, int)` is `True`.
- $1 \leq num_students \leq 100000$
- Each input name contains at least a first name and a last name.
- Names may contain middle names and extra spaces before or after the name.

Function Specification

Write a function named **generate_ids** that takes one parameter, **num_students**. The function must input num_students names and print one generated email ID for each student.

Sample Interaction

```
>>> generate_ids(3)
Harry Potter
hp00000@st.habib.edu.pk
Hermione Granger
hg00001@st.habib.edu.pk
Tom Marvolo Riddle
tr00002@st.habib.edu.pk

>>> generate_ids(4)
Waqar Saleem Dumbledore
wd00000@st.habib.edu.pk
Umair Azfar Khan
uk00001@st.habib.edu.pk
Shahid Snape Hussain
sh00002@st.habib.edu.pk
Shah Jamal Alam
sa00003@st.habib.edu.pk
```

In `'Tom Marvolo Riddle'`, the first name is `'Tom'` and the last name is `'Riddle'`. Therefore, the initials are `'t'` and `'r'`. The middle name is ignored.

The third student's serial number is `2`, which is written using five digits as `00002`.

Testing

To test your function, type:

```
pytest tests/test_q8.py
```